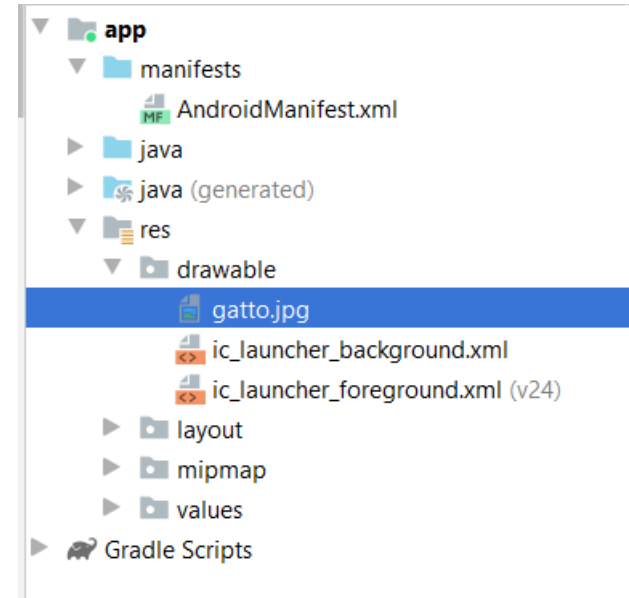


Programmazione Android

Aggiungere le immagini

Aggiungere immagini al progetto

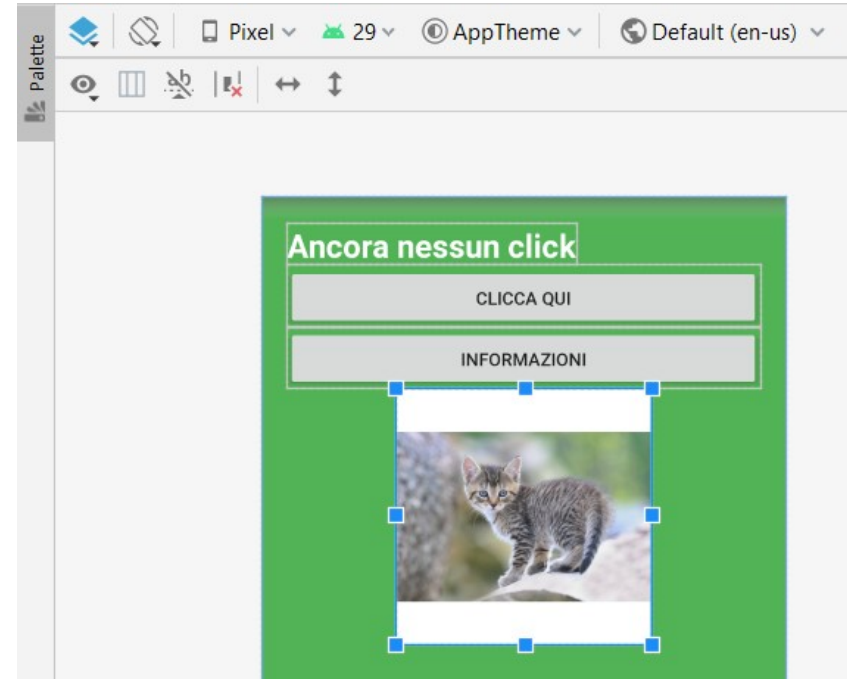
- Le immagini vanno poste nella cartella *res/drawable* (trascinare i file in Android Studio e rilasciarli sulla cartella drawable)
- Il nome del file deve rispettare le **regole dei nomi di variabile in Java...**
- ... quindi non può iniziare con un numero, non può contenere spazi, ecc.
- Aggiungiamo un'immagine chiamata *gatto.jpg* nella posizione mostrata in figura



Aggiungere immagini al progetto

- Modifichiamo il file *activity_main.xml* inserendo il componente ImageView

```
<ImageView  
    android:layout_gravity="center"  
    android:layout_width="200dp"  
    android:layout_height="200dp"  
    android:background="#ffffff"  
    android:src="@drawable/gatto"  
    android:scaleType="centerInside"/>
```



Osservazioni

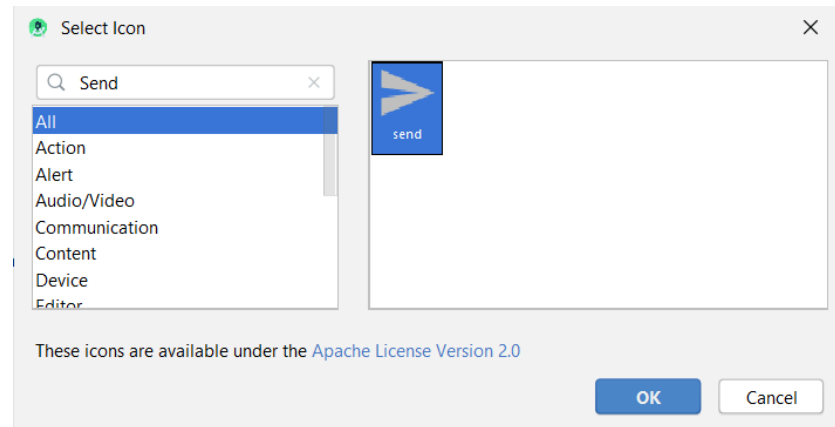
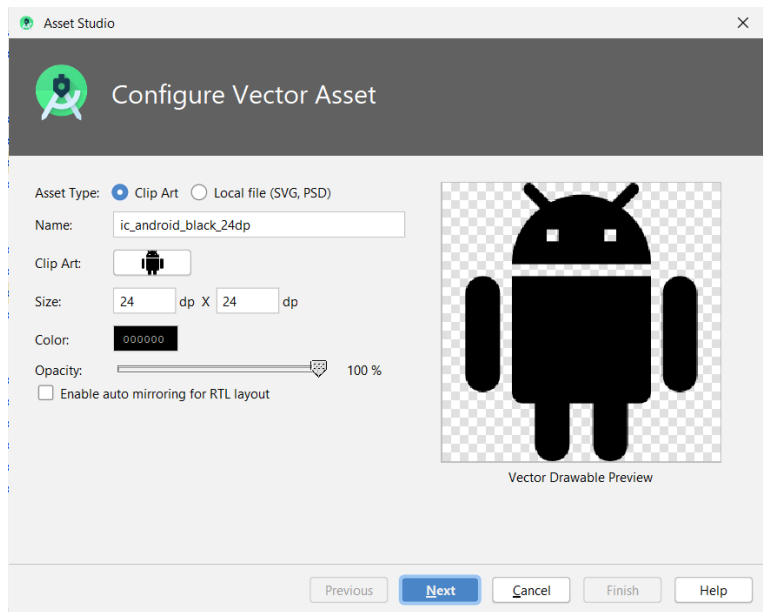
- L'attributo ***layout_gravity*** consente di centrare un componente (in questo caso ImageView) all'interno del suo contenitore (in questo LinearLayout). Questo attributo può essere usato per qualsiasi componente grafico.
- L'attributo `scaleType` è un'attributo specifico di ImageView e consente di definire la modalità di ridimensionamento (scaling) dell'immagine. L'opzione "centerInside" è il valore di default, quindi avremmo potuto omettere l'intera riga.
- Per conoscere le altre opzioni di `scaleType` è sufficiente cancellare il valore e premere CTRL+barra spaziatrice in Android Studio.

Aggiungere icone

- Google mette a disposizione degli sviluppatori un set di icone gratuito noto come ***material icons*** (<https://material.io/resources/icons>)
- Le icone possono essere scaricate in formato png a varie risoluzioni e copiate manualmente nella cartella drawable
- E' tutta via preferibile importare le icone in formato vettoriale seguendo i passaggi seguenti

Importare icone in formato vettoriale

- Dal menu *File* scegliamo *New | Vector Asset*
- Clicchiamo sull'icona a fianco di "Clip Art"
- Cerchiamo l'icona "send" e terminiamo l'importazione

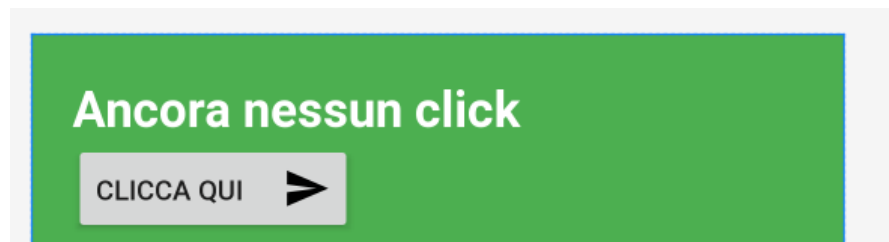


Importare icone in formato vettoriale

- Nella cartella drawable sarà comparso il file ***ic_send_black24dp.xml***. Il prefisso "ic" sta per "icon" (per convenzione). Il formato del file è simile a svg (ma non identico)
- Ora possiamo mostrare l'icona nel primo pulsante modificando il codice del file *activity_main.xml* :

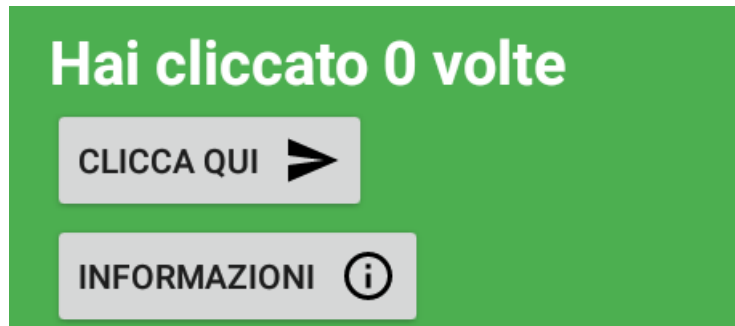
<Button

```
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="CLICCA QUI"  
    android:onClick="pressioneTasto1"  
    android:drawableRight="@drawable/ic_send_black_24dp"  
    android:drawablePadding="10dp"/>
```



Importare icone in formato vettoriale

- In modo simile si può aggiungere la seconda icona, ottenendo:



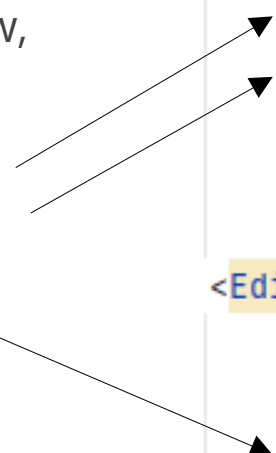
Convertitore Euro/dollaro

Convertitore

- Creare un nuovo progetto "Convertitore"
- Nell'Activity_main.xml aggiungere 2 TextView e 2 EditText (con Id: textView, textView2, editText ed editText2)
- Due TextView per scrivere USD e Euro
- Due EditText per inserire i valori da convertire
- Usare l'elemento LinearLayout

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="USD"
    android:id="@+id/textView"
    android:textSize="25dp"
    android:textStyle="bold"
    android:textColor="#ffffff"/>

<EditText
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/editText"
    android:textSize="25dp"
    android:text="0"/>
```

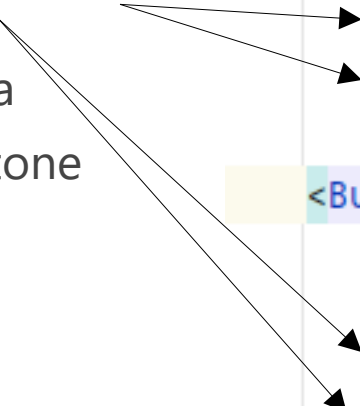


Convertitore

- Nell'Activity_main.xml aggiungere 2 Button (con Id: button e button2)
- Aggiungere il nome del metodo da richiamare quando si preme il bottone

```
<Button
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="USD --> EURO"
    android:id="@+id/button"
    android:onClick="usdToEuro"/>

<Button
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="EURO --> USD"
    android:id="@+id/button2"
    android:onClick="euroToUsd"/>
```



Nel MainActivity-java

- Aggiungere gli import necessari

```
import android.os.Bundle;  
import android.view.View;  
import android.widget.*;  
  
import androidx.activity.EdgeToEdge;  
import androidx.appcompat.app.AppCompatActivity;
```

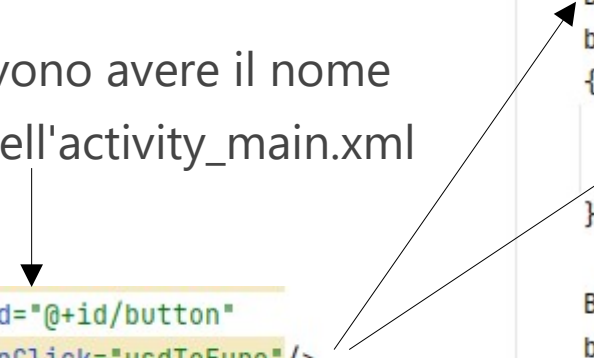
Nel MainActivity.java

- Scrivere i due metodi necessari per calcolare la conversione dollaro/euro e euro/dollaro.
- Attenzione a prendere il valore inserito nel campo EditText e scrivere il risultato con setText

```
public void usdToEuro(View v) {  
    EditText edit_usd=(EditText)findViewById(R.id.editText);  
    EditText edit_euro=(EditText)findViewById(R.id.editText2);  
    double usd=Double.parseDouble(edit_usd.getText().toString());  
    double euro=usd/cambioEuroDollaro;  
    edit_euro.setText(String.valueOf(euro));  
}  
2 usages  
public void euroToUsd(View v) {  
    EditText edit_usd=(EditText)findViewById(R.id.editText);  
    EditText edit_euro=(EditText)findViewById(R.id.editText2);  
    double euro=Double.parseDouble(edit_euro.getText().toString());  
    double usd=euro*cambioEuroDollaro;  
    edit_usd.setText(String.valueOf(usd));  
}
```

Listener

- Aggiungere i metodi listener da eseguire alla pressione dei pulsanti "button" e button2"
- I metodi devono avere il nome assegnato nell'activity_main.xml



```
android:id="@+id/button"  
android:onClick="usdToEuro"/>
```

```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    EdgeToEdge.enable(this);  
    setContentView(R.layout.activity_main);  
  
    Button button_usdToEuro=(Button) findViewById(R.id.button);  
    button_usdToEuro.setOnClickListener(new View.OnClickListener()  
    {  
        public void onClick(View v)  
        { usdToEuro(v);}   
    });  
  
    Button button_euroToUsd=(Button) findViewById(R.id.button2);  
    button_euroToUsd.setOnClickListener(new View.OnClickListener()  
    {  
        public void onClick(View v)  
        { euroToUsd(v);  
        }  
    });  
}
```

Listener

- Aggiungere i metodi listener da eseguire alla pressione dei pulsanti "button" e button2"
- I metodi devono avere il nome assegnato nell'activity_main.xml

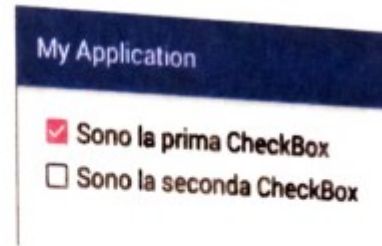
```
android:id="@+id/button"  
android:onClick="usdToEuro"/>  
  
android:id="@+id/button2"  
android:onClick="euroToUsd"/>
```

```
public void usdToEuro(View v) {  
    EditText edit_usd=(EditText)findViewById(R.id.editText);  
    EditText edit_euro=(EditText)findViewById(R.id.editText2);  
    double usd=Double.parseDouble(edit_usd.getText().toString());  
    double euro=usd/cambioEuroDollaro;  
    edit_euro.setText(String.valueOf(euro));  
}  
  
1 usage  
public void euroToUsd(View v) {  
    EditText edit_usd=(EditText)findViewById(R.id.editText);  
    EditText edit_euro=(EditText)findViewById(R.id.editText2);  
    double euro=Double.parseDouble(edit_euro.getText().toString());  
    double usd=euro*cambioEuroDollaro;  
    edit_usd.setText(String.valueOf(usd));  
}
```

Altri widget

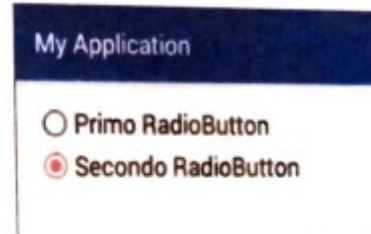
CheckBox

Rappresenta un controllo di tipo **casella a scelta multipla** ed è presente nella classe `android.widget.CheckBox`. Estende la classe `Button` e la classe `TextView`, infatti possiamo anche in questo controllo impostare il testo mostrato a fianco delle caselle di spunta, attraverso il metodo `setText()`. Il metodo `isChecked()` restituisce `true` o `false` a seconda che il controllo venga selezionato.



RadioButton

Rappresenta un controllo di tipo **casella a scelta esclusiva** ed è presente nella classe `android.widget.RadioButton`. Estende la classe `Button` e la classe `TextView`, infatti possiamo anche in questo controllo impostare il testo mostrato a fianco delle caselle di spunta, attraverso il metodo `setText()`. Il metodo `isChecked()` restituisce `true` o `false` a seconda che il controllo venga selezionato. Questi controlli possono essere raggruppati all'interno di un `RadioGroup` in modo che l'utente possa così attivare una sola delle opzioni del gruppo.



Altri widget

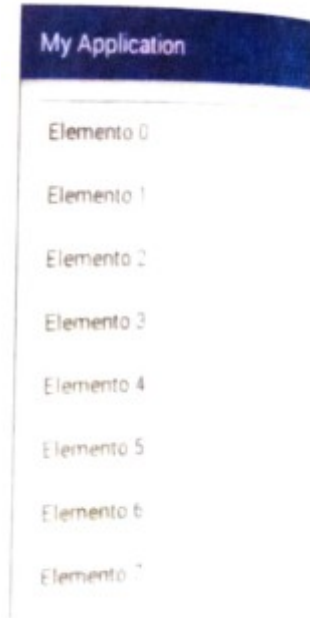
ToggleButton

Rappresentano degli **interruttori** toccabili sullo schermo e possiedono due stati: attivo (on) e non attivo (off). Sono presenti nella classe `android.widget.ToggleButton`. Lo stato è indicato dal metodo `isChecked()`, che restituisce `true` o `false` a seconda se il pulsante sia in stato on oppure off.



ListView

Rappresentano un controllo per agevolare la visualizzazione di numerosi elementi dello stesso tipo. È un controllo presente nella classe `android.widget.ListView`. Per riempirlo bisogna inizializzare un array di stringhe, creare un `arrayadapter`, e chiamare il metodo `setAdapter()` passando come parametro l'`arrayadapter` precedentemente creato.



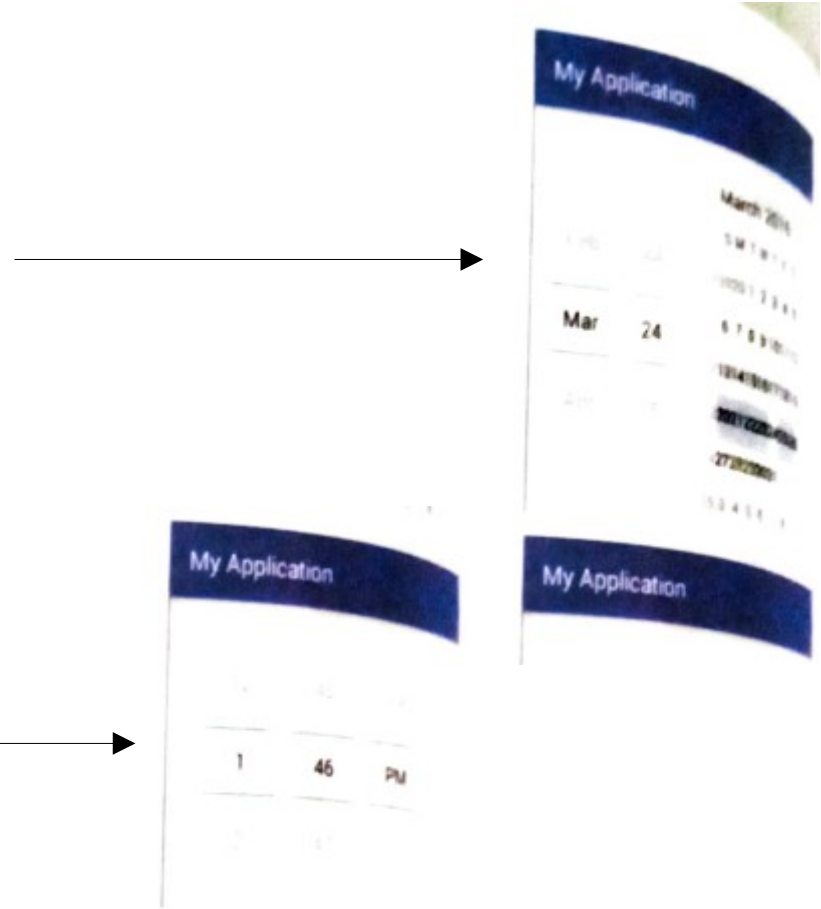
Altri widget

DatePicker

Rappresentano un controllo per agevolare l'utente nell'immissione di una data nel formato giorno, mese, anno. È un controllo presente nella classe `android.widget.DatePicker`. Possiamo recuperare la data impostata dall'utente grazie ai metodi `getDayOfMonth()`, `getMonth()` e `getYear()`.

TimePicker

Rappresentano un controllo per agevolare l'utente nell'immissione di un orario nel formato ore, minuti. È un controllo presente nella classe `android.widget.TimePicker`. Possiamo recuperare l'orario impostato dall'utente grazie ai metodi `getCurrentHour()` e `getCurrentMinute()`.



Esercizio: generatore di riassunto

Esercizio

- Realizzare un'app che consenta all'utente di inserire dei dati (nome, cognome, sesso, data di nascita e campi d'interesse). L'applicazione deve poter generare un riassunto in formato testo delle informazioni inserite dall'utente.
- Widget da utilizzare:
 - EditText per inserire nome e cognome
 - RadioGroup con 2 RadioButton per inserire il sesso
 - DatePicker per la data di nascita
 - CheckBox per i campi di interesse

The screenshot shows an Android application interface with a purple header bar. The form is set against a green background and contains the following elements:

- Nome:** A text field containing "AAAAAA".
- Cognome:** A text field containing "BBBBBB".
- DataNascita:** A date picker showing "Dec 07 2018". Below it, two other dates are visible: "Jan 08 2019" and "Feb 09 2020".
- Sesso:** Two radio buttons labeled "F" and "M". The "M" button is selected.
- Interessi:** Three checkboxes labeled "Tecnologia", "Arte", and "Sport". All are currently unchecked.
- Genera riassunto:** A purple button at the bottom of the form.
- Summary:** Below the button, the collected data is displayed: "Nome: AAAAAA", "Cognome: BBBBBB", "Data di nascita: 8/1/2019", "Interessi:", and "Sesso: M".

Esercizio

- Quando si preme il bottone "genera riassunto" bisogna leggere i valori dei vari widget e utilizzarli per generare il riepilogo delle informazioni in formato testo.
- Metodi per leggere dai widget:
 - EditText ---> getText() restituisce il testo
 - CheckBox ---> isChecked() restituisce true se la casella è spunta
 - RadioGroup ---> isCheckedRadioButtonId() restituisce l'id del radio button selezionato all'interno del gruppo
 - DatePicker ---> getYear() restituisce l'anno, getMonth() restituisce il mese (partendo da zero) e getDayOfMonth() restituisce il numero del giorno

Activity main xml

<DatePicker

```
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/datePicker"
    android:textSize="15dp"
    android:datePickerMode="spinner"
    android:calendarViewShown="false"
/>
```

<CheckBox

```
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/checkBoxTecnologia"
    android:textSize="15dp"
    android:text="Tecnologia"/>
```

<RadioButton

```
    android:id="@+id/F"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:checked="false"
    android:padding="4dp"
    android:text="F"
    android:textSize="15dp" />
```

<TextView

```
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Riassunto"
    android:id="@+id/textViewRiassunto"
    android:textSize="15dp"
    android:textStyle="bold"
    android:textColor="#ffffff"/>
```

<Button

```
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Genera riassunto"
    android:id="@+id/button"
    android:onClick="generaRiassunto"/>
```

Main activity java

```
public class MainActivity extends AppCompatActivity {
```

2 usages

```
    EditText nome=null, cognome=null;
```

2 usages

```
    RadioButton sessof=null, sessom=null;
```

4 usages

```
    DatePicker data=null;
```

2 usages

```
    CheckBox tecnologia=null, arte=null, sport=null;
```

2 usages

```
    TextView riassunto=null;
```

```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    // EdgeToEdge.enable(this);  
    setContentView(R.layout.activity_main);  
    nome=(EditText) findViewById(R.id.editTextNome);  
    cognome=(EditText) findViewById(R.id.editTextCognome);  
    data=(DatePicker) findViewById(R.id.datePicker);  
    tecnologia=(CheckBox) findViewById(R.id.checkBoxTecno);  
    arte=(CheckBox) findViewById(R.id.checkBoxArte);  
    sport=(CheckBox) findViewById(R.id.checkBoxSport);  
    sessof=(RadioButton) findViewById(R.id.F);  
    sessom=(RadioButton) findViewById(R.id.M);  
  
    riassunto = (TextView) findViewById(R.id.textViewRiassunto);  
}
```

Main activity java

```
public void generaRiassunto(View v) {  
    String s= "Nome: "+nome.getText().toString()+"\n"  
        + "Cognome: "+cognome.getText().toString()+"\n"  
        + "Data di nascita: "+data.getDayOfMonth()  
        + "/" +(data.getMonth()+1)  
        + "/" +data.getYear()+"\n"  
        + "Interessi: "+" \n";  
    if(tecnologia.isChecked()) s+="tecnologia"+" \n";  
    if(arte.isChecked()) s+="arte"+" \n";  
    if(sport.isChecked()) s+="sport"+" \n";  
    if(sesoF.isChecked()) s+="Sesso: F"+" \n";  
    if(sesoM.isChecked()) s+="Sesso: M"+" \n";  
    riassunto.setText(s);  
}
```

Esercizio

Scrivere un'app che legge un numero arabo e lo converte in numero romano

Inserire il numero arabo [1-3999]

2389

Numero romano

MMCCCLXXXIX

RESET

Suggerimento:

```
String[] m = {"", "M", "MM", "MMM"} ;  
String[] ce = {"", "C", "CC", "CCC", "CD", "D", "DC", "DCC", "DCCC", "CM"};  
String[] d = {"", "X", "XX", "XXX", "XL", "L", "LX", "LXX", "LXXX", "XC"};  
String[] u = {"", "I", "II", "III", "IV", "V", "VI", "VII", "VIII", "IX"};
```